Trabajo Práctico 2: Punto Fijo

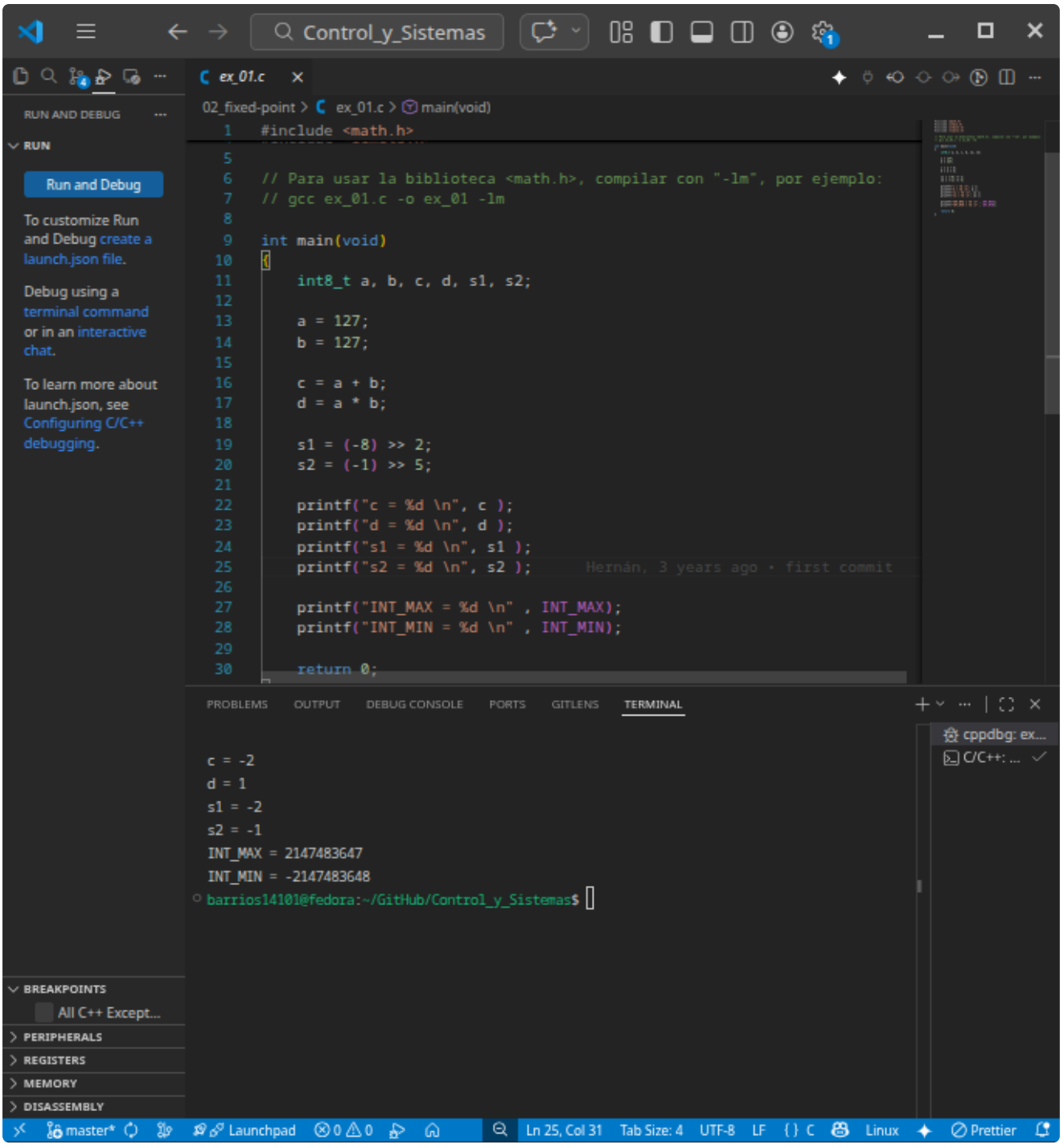
Ejercicio 1

Consigna

Compile el siguiente código en C en su PC:

```c
#include <math.h>
#include <stdio.h>

int main(void){

int8_t a, b, c, d, s1, s2;

a = 127;
b = 127;
c = a + b;
d = a * b;
s1 = (-8) >> 2;
s2 = (-1) >> 5;
printf("c = %d \n", c );
printf("d = %d \n", d );
printf("s1 = %d \n", s1 );
printf("s2 = %d \n", s2 );

}
```

1. Verifique el valor de las variables `c` y `d`. ¿Son estos valores correctos?
2. De no ser así ¿Qué soluciones propone?
3. Verifique el valor de las variables `s1` y `s2` ¿Son estos valores correctos?



Los valores de `c` y `d` son incorrectos.

Aumentar el tamaño de los registros de las variables solucionaría este problema. La variable `int8_t` tiene $2^8 - 1$ posibles números, los cuales están en el rango $[-128, 127]$ (donde se cuenta el 0).

Si se tuviera `a=127` y `b=1` el resultado igualmente sería erróneo (`c=-128`).

Si el **hardware** lo permite, utilizar `int16_t`. Sino, se deberá cambiar el hardware para permitir utilizar esta tamaño de variable.

Las variables `s1` y `s2` son de 8 bits. La acción `>>` es un *shift derecho*.

$$-8_{10} = 1111\ 1000_2$$

```
1111 1000
```

Muevo 2 lugares, sin contar el primer número que es el de signo

```
1111 1000
1111 1100
1111 1110
```

Y pasando a decimal:

$$1111\ 1110_2 = -2_{10}$$

Idem para `s2 = (-1) >> 5`

$$-1_{10} = 1111\ 1111_2$$

```
1111 1111
```

Muevo 2 lugares, sin contar el primer número que es el de signo

```
1111 1111
1111 1111
1111 1111
```

Y pasando a decimal:

$$1111\ 1111_2 = -1_{10}$$

Ejercicio 2

Consigna

Cree 2 funciones en C:

1. Una función para pasar de punto fijo a punto flotante `fx2fp()`.
2. Una función para pasar de punto flotante a punto fijo `fp2fx( )`.
3. Verifique el correcto funcionamiento haciendo

```
b = fx2fp( fp2fx( 2.4515) )
```

4. Compare b con 2.4515 para $Q_{23,8}$ y $Q_{21,10}$.

```c
#include <stdio.h>
#include <stdint.h>
#include <math.h>

int32_t fp2fx(double x, int n) // Convierte un número de punto flotante a
formato fijo Qm.n
{
    return (int32_t)(x * (1 << n));
}

double fx2fp(int32_t x_fx, int n) // Convierte un número de formato fijo
Qm.n a punto flotante
{
    return (double)x_fx / (double)(1 << n);
}


int main(void)
{
    double x = 2.4515; //valor de prueba

    double b_q23_8, b_q21_10;  // Variables para almacenar los valores
convertidos de formato fijo a punto flotante

    int32_t fx_q23_8, fx_q21_10;  // Variables para almacenar los valores
convertidos de punto flotante a formato fijo

    // Q23.8
    fx_q23_8 = fp2fx(x, 8);
    b_q23_8 = fx2fp(fx_q23_8, 8);

    // Q21.10
    fx_q21_10 = fp2fx(x, 10);
    b_q21_10 = fx2fp(fx_q21_10, 10);

    printf("Valor original: %.10f\n\n", x);
    pritf("Q23.8:\n");
    printf("  fijo = %d\n", fx_q23_8);
    printf("  b = %.10f\n", b_q23_8);
    printf("  error = %.10f\n\n", b_q23_8 - x);

    printf("Q21.10:\n");
    printf("  fijo = %d\n", fx_q21_10);
```

```
    printf("  b = %.10f\n", b_q21_10);
    printf("  error = %.10f\n", b_q21_10 - x);

    return 0;
}
```

Formato $Q_{23,8}$

Precision:

$$2^{-8} = 0.00390625$$

Punto Fijo: 627

$$(627)_{10} = (001001110011)_2$$

En formato Q23.8:

$$00000000000000000000010.\,01110011$$

El bit de signo es 0, por lo que el valo en punto flotante es positivo.
Valor en punto flotante:

$$1 * 2^1 + 1 * 2^{-2} + 1 * 2^{-3} + 1 * 2^{-4} + 1 * 2^{-7} 1 * 2^{-8} = 2.44921875$$

Error:

$$E = 2.44921875 - 2.4515 = -0.002812500$$

Formato $Q_{21,10}$

Precision:

$$2^{-10} = 0.0009765625$$

Punto Fijo: 627

$$(2510)_{10} = (100111001110)_2$$

En formato Q23.8:

$$0000000000000000000010.0111001110$$

Valor en punto flotante:

$$1 \times 2^1 + 1 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-7} + 1 \times 2^{-8} + 1 \times 2^{-9} = 2.451171875$$

A diferencia del formato anterior, ahora la cuenta tiene un termino 2^{-9}, lo cual hace que el error respecto al valor original disminuya.

$$E = 2.451171875 - 2.4515 = -0.0003281250$$

Se puede concluir que al usar un formato de punto fijo con 10 bits para la parte fraccionaria, se obtiene mejor precision en la representación de valores decimales que

al usar solo 8 bits para la parte fraccionaria. El error con respecto al valor de prueba disminuyó en un orden de magnitud.

Ejercicio 3

> Consigna

Multiplique dos números en $Q_{21,10}$. El resultado debe ser ajustado según los dos esquemas de redondeos estudiados.

1. Cree una función que implemente redondeo por truncación. `int32_t truncation(int64_t X)`
2. Cree una función que implemente redondeo al valor más cercano. `int32_t rounding(int64_t X)`
3. Compare el resultado de cada esquema de redondeo con el resultado que obtendría usando números en formato double.

Los números en $Q_{21,10}$ están contenidos en $[-2^{21}, 2^{21}-1] = [-2097152, 2097151]$. Luego, $Q_{21,10}$ abarca un intervalo similar $[-2097152, 2097150.9990234375]$.

El código fuente es el siguiente:

```
#include <stdint.h>
#include <stdio.h>

// Notación Qm.n: m bits para la parte entera, n bits para la parte
fraccionaria

#define n 10  // Tu resolución actual
#define fp2fx(f) ((int32_t)((f) * (1 << n)))
#define fx2fp(q) ((float)(q) / (float)(1 << n))

int32_t truncation(int64_t X){
    return (int32_t)(X >> n);
}

int32_t rounding(int64_t X){
    return truncation(X + (1 << (n - 1)));
}

void calcular(double num1, double num2){

    int32_t num1_Q21_10 = fp2fx(num1);
    int32_t num2_Q21_10 = fp2fx(num2);
```

```
    int64_t product_Q21_10 = (int64_t)(num1_Q21_10 * num2_Q21_10);
    double product = num1 * num2;

    int32_t product_trunc = truncation(product_Q21_10);
    int32_t product_round = rounding(product_Q21_10);

    char buffer[100]; // Buffer temporal para armar cada línea

    printf("╔═══════════════════════════════════════════════════╗\n");

    printf("│ %-52s │\n", "Valores Reales:");
    sprintf(buffer, "  num1: %.11f", num1); printf("│ %-52s │\n", buffer);
    sprintf(buffer, "  num2: %.11f", num2); printf("│ %-52s │\n", buffer);

    printf("│ %-52s │\n", "Resultados:");

    sprintf(buffer, "   (double) num1*num2: %.11f", product);
    printf("│ %-52s │\n", buffer);

    sprintf(buffer, "   truncation(num1*num2): %.11f",
fx2fp(product_trunc));
    printf("│ %-52s │\n", buffer);

    sprintf(buffer, "   rounding(num1*num2): %.11f",
fx2fp(product_round));
    printf("│ %-52s │\n", buffer);

    printf("╚═══════════════════════════════════════════════════╝\n");
}


int main(void){

    // Se pasa del rango de Q21.11
    calcular(2000000, 10);
    // Multiplicación de números negativos
    calcular(-1, -10);
    // Multiplicación con 10 números después del punto decimal
    calcular(1.0000000001, 2);
    // Multiplicación dentro del rango de Q21.11
    calcular(100, 48);
    // Multiplicación con 2 números con 5 dígitos después del punto
```

```
decimal
    calcular(1.11111, 2.55555);
    // Multiplicación arbitraria
    calcular(46.1948, 781.4874);

    return 0;
}
```

Y los resultados son:

```
╔═══════════════════════════════════════════════════╗
║ Valores Reales:                                   ╒
╒   num1: 2000000.00000000000                       ╒
╒   num2: 10.00000000000                            ╒
╒ Resultados:                                       ╒
╒    (double) num1*num2: 20000000.00000000000       ╒
╒    truncation(num1*num2): -768.00000000000        ╒
╒    rounding(num1*num2): -768.00000000000          ╒
╚═══════════════════════════════════════════════════╝

╔═══════════════════════════════════════════════════╗
║ Valores Reales:                                   ╒
╒   num1: -1.00000000000                            ╒
╒   num2: -10.00000000000                           ╒
╒ Resultados:                                       ╒
╒    (double) num1*num2: 10.00000000000             ╒
╒    truncation(num1*num2): 10.00000000000          ╒
╒    rounding(num1*num2): 10.00000000000            ╒
╚═══════════════════════════════════════════════════╝

╔═══════════════════════════════════════════════════╗
║ Valores Reales:                                   ╒
╒   num1: 1.00000000010                             ╒
╒   num2: 2.00000000000                             ╒
╒ Resultados:                                       ╒
╒    (double) num1*num2: 2.00000000020              ╒
╒    truncation(num1*num2): 2.00000000000           ╒
╒    rounding(num1*num2): 2.00000000000             ╒
╚═══════════════════════════════════════════════════╝

╔═══════════════════════════════════════════════════╗
║ Valores Reales:                                   ╒
╒   num1: 100.00000000000                           ╒
╒   num2: 48.00000000000                            ╒
╒ Resultados:                                       ╒
```

```
    (double) num1*num2: 4800.00000000000
    truncation(num1*num2): 704.00000000000
    rounding(num1*num2): 704.00000000000

Valores Reales:
  num1: 1.11111000000
  num2: 2.55555000000
Resultados:
    (double) num1*num2: 2.83949716050
    truncation(num1*num2): 2.83593750000
    rounding(num1*num2): 2.83691406250

Valores Reales:
  num1: 46.19480000000
  num2: 781.48740000000
Resultados:
    (double) num1*num2: 36100.65414552000
    truncation(num1*num2): -763.71386718750
    rounding(num1*num2): -763.71289062500
```

Es interesante notar como:

- Al pasarse del número 2097150.9990234375, el resultado se vuelve erróneo.
- Se pierde información por "underflow" (3er caso).
- Aunque los números reales parecen chicos (46 y 781), al redefinirlos como variables de punto fijo, vemos que se convierten en enteros gigantescos. La representación entera (Q_{10}):
  - $a \approx 46.1948 \times 1024 = \mathbf{47303}$
  - $b \approx 781.4874 \times 1024 = \mathbf{800243}$
  - $47303 \times 800243 = \mathbf{37.854.194.629}$.

Ejercicio 4

Consigna

Implemente la suma de dos números con aritmética de saturación,

```
int32_t saturation(int32_t a, int32_t b)
```

1. Utilice las constantes INT_MAX e INT_MIN definidas en la biblioteca limits.h para la
2. Opere con una serie de número para verificar que saturation() funciona correctamente.

```
#include <stdio.h>
#include <stdint.h>
#include <limits.h>

int32_t saturation(int32_t a, int32_t b)
{
    int64_t sum = (int64_t)a + (int64_t)b; // Usamos int64_t para evitar
overflow

    if (sum > INT32_MAX) {
        return INT32_MAX; // Saturación positiva
    } else if (sum < INT32_MIN) {
        return INT32_MIN; // Saturación negativa
    } else {
        return (int32_t)sum; // No hay saturación, retornamos la suma
normal
    }
}

int main(void)
{
    int32_t a, b, r;

    printf("Prueba normal:\n");
    a = 100;
    b = 200;
    r = saturation(a, b);
    printf("%d + %d = %d\n\n", a, b, r);

    printf("Overflow positivo:\n");
    a = INT_MAX - 10;
    b = 50;
    r = saturation(a, b);
    printf("%d + %d = %d\n\n", a, b, r);

    printf("Overflow negativo:\n");
    a = INT_MIN + 10;
    b = -50;
```

```
    r = saturation(a, b);
    printf("%d + %d = %d\n\n", a, b, r);

    printf("Caso límite:\n");
    a = INT_MAX;
    b = 1;
    r = saturation(a, b);
    printf("%d + %d = %d\n", a, b, r);

    return 0;

}
```

Se plantea una suma de enteros de 32 bits con signo. El rango de valores que se pueden representar con 32 bits es:

$$\left[-2^{31}, 2^{31}-1\right] = \left[-2\,147\,483\,648\ ,\ 2\,147\,483\,647\right]$$

En el primer caso de prueba se intenta sumar un valor que es solo 10 unidades menor al entero máximo que se puede representar (2147483637), más 50.
El valor de la suma es:

$$2147483637 + 50 = 2147283687$$

Sin embargo, lo que se obtiene es el entero máximo, ya que se aplicó la saturación.

$$2147483637 + 50 = 2147283647$$

Se realiza un procedimiento análogo en el segundo caso de prueba: se suma un valor que es 10 unidades mayor al entero mínimo que se puede representar (-214783638), más (-50).
El valor de la suma es:

$$-2147483638 + (-50) = -214728368$$

Sin embargo, el valor que se obtiene es el entero mínimo que es posible representar con 32 bits:

$$-2147483638 + (-50) = -2147283648$$

En el caso de prueba límite, se toma el mayor valor entero que se puede representar en 32 bits (2147483647) y se le suma 1. Se observa que la suma no se efectúa ya que al aplicar saturación el resultado sigue siendo el valor máximo de 32 bits.

$$2147483647 + 1 = 2147483647$$

Ejercicio 5

> Consigna

1. Escriba un programa en C que implemente la operación MAC en punto fijo para los siguientes vectores:

```
float X[5] = {1.1, 2.2, 3.3, 4.4, 5.5};
float Y[5] = {6.6, 7.7, 8.8, 9.9, 10.10};
```

2. Convierta ambos vectores a formato `Q21.10`.
3. Implemente la operación MAC con dos enfoques:
    1. Multiplique 2 números de 32 bits y redondee por truncación antes de sumar cada resultado.
    2. Multiplique 2 números de 64 bits y solo redondee luego de sumar todos los números.
4. Compare ambos resultados en punto fijo con el que se obtiene al operar en formato `double`.
5. Finalmente, compare los valores de `acum_32a`, `acum_32b` y `acum_db`.

Código fuente:

```
#include <stdint.h>
#include <stdio.h>

// Notación Qm.n: m bits para la parte entera, n bits para la parte
fraccionaria
#define n 10  // Tu resolución actual
#define fp2fx(f) ((int32_t)((f) * (1 << n)))
#define fx2fp(q) ((float)(q) / (float)(1 << n))

#define m 20  // Tu resolución actual
#define fp2fx_(f) ((int64_t)((f) * (1 << m)))
#define fx2fp_(q) ((float)(q) / (float)(1 << m))

int32_t truncation(int64_t X){
    return (int32_t)(X >> n);
}

int main(){
    float X[5] = {1.1, 2.2, 3.3, 4.4, 5.5};
    float Y[5] = {6.6, 7.7, 8.8, 9.9, 10.10};

    int32_t X_fx[5] = {0}, Y_fx[5] = {0};

    for(int i = 0; i < 5; i++){
```

```
        X_fx[i] = fp2fx(X[i]);
        Y_fx[i] = fp2fx(Y[i]);
    }

    int32_t acum_32a = 0, acum_32b = 0;
    int64_t acum_64 = 0;

    for(int i = 0; i < 5; i++){
        acum_32a += (int32_t ) ( truncation( (int64_t)(X_fx[i] * Y_fx[i] )
));
        acum_64 += (int64_t) ( (int64_t) (X_fx[i] * Y_fx[i] ) );
    }

    acum_32b += (int32_t) acum_64;

    double dX[5] = {1.1, 2.2, 3.3, 4.4, 5.5};
    double dY[5] = {6.6, 7.7, 8.8, 9.9, 10.10};

    double acum_double = 0.0;

    for(int i = 0; i < 5; i++){
        acum_double += dX[i] * dY[i];
    }

    char buffer[100]; // Buffer temporal para armar cada línea
    printf("╔═══════════════════════════════════════════════════╗\n");

    printf("│ %-52s │\n", "Suma acumulada por truncation:");
    sprintf(buffer, "  >> FX: %d", acum_32a); printf("│ %-52s │\n",
buffer);
    sprintf(buffer, "  >> FP: %.11f", fx2fp(acum_32a)); printf("│ %-52s
│\n", buffer);

    printf("╚═══════════════════════════════════════════════════╝\n");

    printf("╔═══════════════════════════════════════════════════╗\n");

    printf("│ %-52s │\n", "Suma acumulada por acumulación en 32 bits:");
    sprintf(buffer, "  >> FX: %d", acum_32b); printf("│ %-52s │\n",
buffer);
    sprintf(buffer, "  >> FP: %.11f", fx2fp(acum_32b)); printf("│ %-52s
│\n", buffer);
```

```
    printf("╚═══════════════════════════════════════════════════╝\n");


    printf("╔═══════════════════════════════════════════════════╗\n");

    printf("╟ %-52s ╟\n", "Suma acumulada por acumulación en 64 bits:");
    sprintf(buffer, "  >> FX: %d", acum_64); printf("╟ %-52s ╟\n",
buffer);
    sprintf(buffer, "  >> FP: %.11f", fx2fp(acum_64)); printf("╟ %-52s
╟\n", buffer);

    printf("╚═══════════════════════════════════════════════════╝\n");


    printf("╔═══════════════════════════════════════════════════╗\n");

    printf("╟ %-52s ╟\n", "Corrección de interpretación desde Q21.20:");
    sprintf(buffer, "  >> FX: %d", acum_64 / 1024); printf("╟ %-52s ╟\n",
buffer);
    sprintf(buffer, "  >> FP: %.11f", fx2fp(acum_64/1024)); printf("╟
%-52s ╟\n", buffer);

    printf("╚═══════════════════════════════════════════════════╝\n");



    printf("╔═══════════════════════════════════════════════════╗\n");

    printf("╟ %-52s ╟\n", "Suma acumulada con double:");
    sprintf(buffer, "  >> FP: %.11f", acum_double); printf("╟ %-52s ╟\n",
buffer);

    printf("╚═══════════════════════════════════════════════════╝\n");

    return 0;
}
```

Y su resultado:

```
╔═══════════════════════════════════════════════════╗
╟ Suma acumulada por truncation:                    ╟
╟   >> FX: 155980                                   ╟
╟   >> FP: 152.32421875000                          ╟
```

```
╚═══════════════════════════════════════════════╝

╔═══════════════════════════════════════════════╗
╟ Suma acumulada por acumulación en 64 bits:    ╟
╟   >> FX: 159725774                            ╟
╟   >> FP: 155982.20312500000                   ╟
╚═══════════════════════════════════════════════╝

╔═══════════════════════════════════════════════╗
╟ Corrección de interpretación desde Q21.20:    ╟
╟   >> FX: 155982                               ╟
╟   >> FP: 152.32617187500                      ╟
╚═══════════════════════════════════════════════╝

╔═══════════════════════════════════════════════╗
╟ Suma acumulada por acumulación en 32 bits:    ╟
╟   >> FX: 159725774                            ╟
╟   >> FP: 155982.20312500000                   ╟
╚═══════════════════════════════════════════════╝

╔═══════════════════════════════════════════════╗
╟ Suma acumulada con double:                    ╟
╟   >> FP: 152.35000000000                      ╟
╚═══════════════════════════════════════════════╝
```

Se puede observar diferencia significativa en los resultados de la acumulación en 64 bits (Caso B) frente a la referencia en `double`. De ello se puede concluir las siguientes observaciones:

1. **Error de Interpretación de Escala:** En la multiplicación en complemento a 2, al multiplicar dos números en formato Q_{10} ($N = 10$ bits fraccionarios), el resultado posee $2n = 20$ bits fraccionarios (Q_{20}).
   - En el **Caso B**, la suma se realizó en un acumulador de 64 bits manteniendo los 20 bits fraccionarios del producto.
   - El valor obtenido (`155982.203`) es el resultado correcto en escala Q_{20} interpretado erróneamente con una una función que traduce de la escala Q_{10}.
   - Si normalizamos el resultado del Caso B dividiendo por el factor de escala faltante (2^{10}), el valor real es **152.326** (valor más cercano al de la cuenta hecha con `double`).
2. **Error de Cuantización por Truncamiento (Caso A):** En la acumulación de 32 bits, al aplicarse la función de **truncamiento** (`X >> 10`) antes de cada suma, se tiene que:
   - Se normaliza cada producto individual de vuelta a Q_{10}.
   - Aunque el resultado final (`152.324`) es más cercano al resultado con `double`, este posee un mayor **error de cuantización** acumulado respecto debido a la pérdida de los 10 bits menos significativos en cada paso de la sumatoria (truncamiento en cada paso).

3. **Riesgo de Overflow:** Utilizando el debugger, se puede analizar que la operación manual `(int64_t)(X_fx[i] * Y_fx[i])` es peligrosa: el compilador realiza la multiplicación en registros de 32 bits antes de la promoción, lo que causa **rollover** si los valores superan el límite de `INT32_MAX`, invalidando cualquier resultado posterior. El rollover ocurre desde el primer producto, ya que la cuenta real es $1.1 \times 6.6 = 6.6$, mientras que la cuenta que hace la computadora en punto fijo es $1126 \times 6758 = 7609508$ el cual es un número que supera `INT32_MAX` por lo que de allí en adelante se considera erroneo.

Ejercicio 6

> Consigna
>
> Suponga que debe implementar un filtro digital con un procesador DSP de 16 bits. El mismo cuenta con un acumulador de 40 bits en su ALU. Calcule cuántas sumas consecutivas puede realizar garantizando que no se producirá un overflow

Para s sumandos, el acumulador debe tener: $N + log_2(s)$ bits.
Es decir que para un acumulador de 40 bits:

$$40 = 16 + log_2(s)$$

$$24 = log_2(s)$$

$$s = 2^{24} = 16777216$$

Entonces, con un acumulador de 40 bits se pueden sumar hasta 16777216 sumandos, es decir que se pueden realizar 12777215 sumas consecutivas.